

A DevSecOps Security Monitoring Architecture for LogiFlow: From Vulnerable Baseline to Continuous Threat Visibility

Andersson David Sanchez Mendez, Cristian Santiago Pedraza Rodriguez, Jeisson David Sanchez Gomez

Escuela Colombiana de Ingenieria Julio Garavito

SPTI - Seguridad y Privacidad de Tecnologias de la Informacion

Bogota, Colombia

Email: {andersson.sanchez, cristian.pedraza-r, jeisson.sanchez}@escuelaing.edu.co

Abstract—LogiFlow is a real-time microservices platform for fleet routing where service trust, operational continuity, and data integrity are critical. This paper presents a complete DevSecOps security monitoring architecture that compares an intentionally vulnerable baseline against a hardened production-oriented design. We identify attack surfaces in API, WebSocket, webhook, container runtime, and CI/CD supply-chain paths, and prioritize threats through STRIDE and OWASP Top 10 (2021). The proposed architecture integrates shift-left controls (CodeQL, Semgrep, npm audit, GitLeaks, Trivy, OWASP ZAP) with shift-right telemetry (Falco, Prometheus, Loki, Grafana) and distributed tracing (OpenTelemetry, Tempo). Results from test scenarios TC-01 to TC-10 show effective mitigation of high-priority exploit paths such as unauthorized socket access, unauthenticated webhook injection, and request-flood abuse, while improving forensic readiness and triage speed. The resulting artifact provides a reusable blueprint for academic and industry teams implementing continuous security in distributed systems.

Index Terms—DevSecOps, Microservices Security, OWASP Top 10, STRIDE, Runtime Monitoring, Falco, Prometheus, Loki, Grafana, OpenTelemetry, Tempo

I. INTRODUCTION

Modern logistics platforms are increasingly event-driven and distributed, which improves responsiveness but expands attack surface and detection latency. In LogiFlow, critical routing decisions are produced by interconnected services (gateway, real-time channels, optimization, automation, and persistence), making security failures operationally expensive.

This work addresses the SPTI objective of security-by-design with measurable controls by implementing and validating a full DevSecOps security layer over LogiFlow. The contribution is threefold:

- 1) A reproducible baseline-to-hardened architecture comparison with explicit vulnerability-to-control mapping.
- 2) An integrated CI/CD security pipeline that combines SAST, SCA, secrets detection, container scanning, and DAST.
- 3) A continuous monitoring and triage stack that correlates metrics, logs, runtime anomalies, and traces for faster incident response.

II. BACKGROUND

A. OWASP Top 10 and STRIDE

OWASP Top 10 (2021) is used as the vulnerability taxonomy to classify weaknesses in access control, authentication, misconfiguration, outdated dependencies, and monitoring gaps [1]. STRIDE guides threat modeling across spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege.

B. NIST CSF and SGSI Perspective

NIST CSF provides a governance structure through Identify, Protect, Detect, Respond, and Recover functions [2]. This aligns with SGSI-style traceability where each control must map to evidence, validation criteria, and operational ownership.

C. DevSecOps Principles

The architecture follows shift-left plus shift-right security: prevent defects early in CI while maintaining runtime anomaly detection and observability in production-like environments [3].

III. LOGIFLOW SYSTEM DESCRIPTION

A. Service Topology

LogiFlow uses a microservices topology with heterogeneous technologies: NestJS gateway, Socket.IO real-time service, VROOM-based optimizer, Redis, PostgreSQL, Python predictor, and n8n automation.

B. Attack Surface and Trust Boundaries

The most exposed paths are: (i) public API endpoints, (ii) socket room subscriptions, (iii) webhook ingestion, (iv) container runtime behavior, and (v) dependency and secrets supply-chain risks in CI.

TABLE I
CORE LOGIFLOW SERVICES AND SECURITY-RELEVANT ROLE

Service	Tech Stack	Security-Relevant Role
Gateway	NestJS, JWT, Prisma	API boundary, token issuance, webhook protection, security metrics
Realtime	Node.js, Socket.IO, Redis	Vehicle channel delivery, socket auth checks, unauthorized-attempt metrics
Optimizer	Node.js, gRPC, VROOM	Route computation service exposed over gRPC/HTTP metrics
Automation	n8n + Express mock webhook	External event ingestion and orchestration trigger path
Monitoring	Falco, Prometheus, Loki, Grafana, Tempo, OTel Collector	Runtime detection, telemetry correlation, trace triage

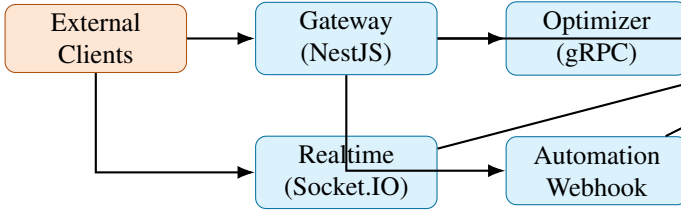


Fig. 1. High-level LogiFlow architecture and security-observable boundaries.

IV. THREAT MODEL

Threats were scored using a simple operational risk equation:

$$R = L \times I \quad (1)$$

where L is likelihood (1–5) and I is impact (1–5).

The mitigation order prioritized scores ≥ 20 before medium-score controls, while preserving traceability to OWASP and NIST CSF.

V. UNSECURE VS SECURE ARCHITECTURE COMPARISON

The secure architecture also improved observability semantics by exposing metrics and traces from gateway and realtime services, reducing *no data* states in dashboards and enabling direct triage links to tracing views.

VI. DEVSECOPS PIPELINE IMPLEMENTATION

A. Pipeline Design

The CI pipeline executes on push, pull request, schedule, and manual dispatch. It combines preventive and detective controls as shown in Fig. 2.



Fig. 2. Security pipeline from shift-left analysis to runtime monitoring.

TABLE II
HIGHEST-PRIORITY THREATS (EXCERPT FROM STRIDE REGISTER)

ID	Threat Scenario	L	I	Score
TM-01	Unauthorized subscription to vehicle socket rooms (Spoofing/Info Disclosure)	5	5	25
TM-02	Forged webhook traffic events alter routing decisions (Spoofing/Tampering)	4	5	20
TM-03	Webhook request flood creates service degradation (DoS)	5	4	20
TM-08	Malicious process execution inside container runtime	4	5	20
TM-11	Exploitation of vulnerable dependencies in exposed service path	4	4	16

B. Implemented Gates and Hardening Details

- **SAST:** CodeQL and Semgrep rulepacks for JavaScript/TypeScript analysis.
- **SCA:** Per-service audit artifacts with criticality evaluation logic.
- **Secrets:** GitLeaks baseline in all push/PR paths.
- **Container Security:** Trivy config scan over gateway Docker context, including non-root requirement remediation.
- **DAST:** OWASP ZAP baseline against gateway health target, with parser-based gating.
- **CI Reliability Hardening:** automatic creation of `logiflow-security-net` in CI before DAST service startup.

VII. CONTINUOUS MONITORING AND TRIAGE STACK

A. Metrics, Logs, Runtime Alerts, and Traces

The monitoring stack includes Falco, Prometheus, Loki, Grafana, Tempo, and OpenTelemetry Collector. Gateway and realtime services export security-oriented metrics and traces.

B. Dashboards and Navigation Improvements

Three security dashboards were stabilized with no-data-safe queries and fast refresh windows. A dedicated traces dashboard was added, plus panel-level links from high-signal panels (Auth Failures and Falco Critical Alerts) directly to Grafana Explore with Tempo preselected. This reduced analyst friction during incident triage.

TABLE III
BASELINE VULNERABILITY TO IMPLEMENTED SECURITY CONTROL MAPPING

ID	Baseline Weakness	Implemented Control	Verification
V-001	Unauthenticated Socket.IO room subscription	JWT middleware enforced before join; unauthorized attempts counted as logiflow_socket_unauthorized_total	Socket injection script returns authentication error
V-002	Webhook endpoint accepts unauthenticated events	Bearer-protected webhook ingestion in gateway and automation flow	Unauthorized requests return HTTP 401
V-003	No rate limiting on traffic-event webhook	Request throttling and rate-limit counters/alerts	Flood scenarios produce HTTP 429
V-005	Missing hardening headers in gateway responses	Helmet + secure bootstrap configuration	Header verification by curl and DAST checks
V-006	Verbose internal error exposure	Production-safe exception handling	Error responses sanitized under production profile
V-007	Dependency exposure in service packages	CI SCA with npm audit policy and artifacts	Critical/high CVE counts monitored per service
V-008	No runtime anomaly detection	Falco rules + Loki/Grafana runtime triage	Suspicious container behavior triggers alerts

TABLE IV
REPRESENTATIVE SECURITY TELEMETRY SIGNALS

Signal	Security Meaning
logiflow_auth_failure	Failed authentication attempts in gateway
logiflow_webhook_request	Inbound webhook request activity
logiflow_socket_unauthorized	Unauthorized socket connection attempts
logiflow_socket_connections	Active real-time sessions by role
Falco critical alerts	Syscall-level runtime anomalies and policy violations
Tempo traces (OTel)	Cross-service execution and forensic timing correlation

TABLE V
VALIDATION SUMMARY FOR CORE SECURITY SCENARIOS

Test	Security Objective	Result
TC-01	Enforce authentication on protected endpoints	Pass
TC-02	Reject unauthorized socket joins	Pass
TC-03	Trigger rate-limit under flood conditions	Pass
TC-04/05	Detect injection/header issues via DAST baseline	Pass (no blocking high findings in hardened target)
TC-06	Prevent leaked credentials in repository	Pass
TC-07	Track dependency vulnerabilities by service	Pass with policy-driven gating
TC-08	Detect suspicious container behavior at runtime	Pass
TC-09	Reject manipulated JWT privilege escalation	Pass
TC-10	Avoid internal error leakage in production mode	Pass

VIII. EVALUATION AND RESULTS

A. Test Plan Coverage

The security test plan defines TC-01 to TC-10 spanning auth bypass, socket abuse, DoS, injection checks, header validation, secrets, dependencies, runtime anomalies, JWT tampering, and error disclosure.

B. Operational Outcomes

Compared to the vulnerable baseline, the hardened architecture provides:

- Stronger prevention for external exploit paths (API, sockets, webhooks).
- Earlier defect detection in CI through layered static and dynamic checks.
- Better runtime detection and faster triage via correlated metrics, logs, and traces.

IX. SPTI COURSE CONCEPTS MAPPING

X. CONCLUSIONS AND FUTURE WORK

This paper demonstrated a full DevSecOps monitoring architecture for LogiFlow that is reproducible, evidence-driven, and aligned with academic security objectives. The transition from a vulnerable baseline to a hardened architecture produced

concrete benefits in preventive controls, runtime detection, and incident triage readiness.

Future work will focus on: (i) automated secret rotation and vault-backed credential governance, (ii) SOAR-style incident orchestration for Falco/Prometheus alerts, (iii) stronger policy-as-code baselines for supply-chain controls, and (iv) longitudinal quantitative benchmarking of mean time to detect and mean time to respond.

ACKNOWLEDGMENT

This work was developed in the context of the SPTI course at Escuela Colombiana de Ingenieria Julio Garavito.

REFERENCES

- [1] OWASP Foundation, "Owasp top 10: The ten most critical web application security risks 2021," <https://owasp.org/www-project-top-ten/>, 2021, accessed: 2026-04-09.

TABLE VI
SPTI CONCEPT-TO-ARTIFACT TRACEABILITY

SPTI Concept	Evidence in This Work	Practical Outcome
Risk Management	STRIDE register with risk equation $R = L \times I$ and prioritization bands	Structured treatment order for highest-impact threats
Security Architecture Principles	JWT guards, bearer enforcement, rate limiting, exception hardening, least-privilege runtime	Reduced attack success probability at trust boundaries
SGSI / Governance	NIST-mapped controls matrix, CI evidence artifacts, repeatable test criteria	Auditability and traceable control ownership
Data Protection	Secrets detection, controlled token flows, reduced unauthorized telemetry exposure	Improved confidentiality and accountability
Software Security	CodeQL, Semgrep, Trivy, ZAP, dependency audit, runtime alerts	Continuous security quality gates and faster remediation
Continuous Monitoring	Falco + Prometheus + Loki + Grafana + Tempo + OTel collector	End-to-end incident detection and correlated investigation

- [2] National Institute of Standards and Technology, “Nist cybersecurity framework (csf) 2.0,” <https://www.nist.gov/cyberframework>, 2024, accessed: 2026-04-09.
- [3] —, “Secure software development framework (ssdf) v1.1,” <https://csrc.nist.gov/pubs/sp/800/218/final>, 2022, accessed: 2026-04-09.